

Travaux Dirigés

Création d'APIs avec Python

API REST · API GraphQL · Dockerisation · Tests	
Cours	ECHE834 — Solutions d'échanges inter-applicatifs
Durée	4 heures (+ travail autonome)
Prérequis	Bases Python, notions HTTP

Table des matières

Table des matières.....	2
1. Introduction et objectifs pédagogiques.....	4
1.1 Objectifs pédagogiques.....	4
1.2 Compétences visées (référentiel RNCP 36921)	4
1.3 Organisation du TD.....	4
2. Setup de l'environnement de développement.....	5
2.1 Installation de Python 3.x.....	5
2.2 Création d'un environnement virtuel.....	5
2.3 Installation des dépendances.....	5
2.4 Structure du projet recommandée	6
2.5 Configuration de l'IDE	6
3. Exercice 1 — API REST avec Flask/FastAPI.....	7
3.A — Exemple guidé : Gestion d'une bibliothèque (Flask)	7
Fichier : exercice1_rest/app.py	7
Démarrage et test rapide.....	10
3.B — Exercice pratique : Créez votre propre API REST	11
3.C — Tests avec pytest.....	12
Fichier : exercice1_rest/tests/test_api.py.....	12
Lancer les tests	13
3.D — Critères d'évaluation (Exercice 1)	14
4. Exercice 2 — API GraphQL avec Strawberry	15
4.A — Exemple guidé : Gestion de livres (Strawberry + FastAPI)	15
Fichier : exercice2_graphql/schema.py	15
Fichier : exercice2_graphql/app.py	17

Démarrage et exemples de requêtes GraphQL	18
4.B — Exercice pratique : Créez votre API GraphQL	19
4.C — Tests d'une API GraphQL	19
4.D — Critères d'évaluation (Exercice 2)	21
5. Guide de Dockerisation	22
5.1 Concepts clés Docker	22
5.2 Dockerfile — API REST (Flask)	22
5.3 Dockerfile — API GraphQL (FastAPI)	23
5.4 docker-compose.yml — Orchestration des deux APIs	23
5.5 Commandes Docker essentielles	24
6. Bonnes pratiques générales	26
6.1 Architecture du projet	26
6.2 Gestion des configurations	26
6.3 Sécurité	27
6.4 Documentation automatique (Swagger/OpenAPI)	27
6.5 Logging et monitoring	27
7. Annexes	29
7.1 Ressources complémentaires	29
7.2 Commandes de référence rapide	29
7.3 Troubleshooting — Problèmes courants	29
7.4 Exemple de fichier .env	30

1. Introduction et objectifs pédagogiques

1.1 Organisation du TD

Le TD est structuré en deux exercices principaux suivis d'un guide de dockerisation :

Section	Durée	Contenu
Exercice 1	~1h30	Création d'une API REST (Flask/FastAPI) — exemple guidé + exercice
Exercice 2	~1h30	Création d'une API GraphQL (Strawberry) — exemple guidé + exercice
Guide Docker	~30 min	Dockerisation des deux APIs
Tests	~30 min	Tests unitaires et d'intégration avec pytest

2. Setup de l'environnement de développement

Avant de commencer les exercices, assurez-vous que votre environnement de développement est correctement configuré.

2.1 Installation de Python 3.x

Vérifiez que Python 3.10 ou supérieur est installé sur votre machine :

```
# Vérification de la version Python
python3 --version
# Résultat attendu : Python 3.10.x ou supérieur

# Si Python n'est pas installé (Ubuntu/Debian)
sudo apt update && sudo apt install python3 python3-pip python3-venv

# Sur macOS avec Homebrew
brew install python3
```

2.2 Création d'un environnement virtuel

L'utilisation d'un environnement virtuel isole les dépendances du projet :

```
# Créer un dossier pour le projet
mkdir td_api_eche834 && cd td_api_eche834

# Créer l'environnement virtuel
python3 -m venv venv

# Activer l'environnement virtuel
# Sur Linux/macOS :
source venv/bin/activate

# Sur Windows (PowerShell) :
.\venv\Scripts\Activate.ps1

# Vérifier que l'environnement est actif (le prompt affiche (venv))
(venv) $ python --version
```

2.3 Installation des dépendances

Créez un fichier requirements.txt à la racine du projet :

```
# requirements.txt
flask==3.0.3
flask-restful==0.3.10
fastapi==0.111.0
uvicorn[standard]==0.30.1
pydantic==2.7.3
strawberry-graphql[fastapi]==0.235.0
pytest==8.2.2
```

```
pytest-asyncio==0.23.7
httpx==0.27.0
python-dotenv==1.0.1
requests==2.32.3
```

Installez les dépendances :

```
pip install -r requirements.txt

# Vérification
pip list
```

2.4 Structure du projet recommandée

Organisez votre projet de la façon suivante :

```
td_api_eche834/
├── venv/
├── exercice1_rest/
│   ├── app.py           # Application Flask/FastAPI
│   ├── models.py        # Modèles de données
│   ├── routes.py        # Définition des routes
│   └── tests/
│       └── test_api.py   # Tests pytest
├── exercice2_graphql/
│   ├── app.py           # Application GraphQL
│   ├── schema.py        # Schéma GraphQL
│   └── tests/
│       └── test_graphql.py
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── .env                 # Variables d'environnement
└── requirements.txt
```

2.5 Configuration de l'IDE

 **IDE recommandé** : VS Code avec les extensions Python (Microsoft), Pylance, REST Client et Thunder Client pour tester les APIs directement depuis l'éditeur.

Extensions VS Code utiles :

- Python (Microsoft) — IntelliSense et débogage
- Pylance — analyse de type avancée
- Thunder Client — client REST intégré
- Docker — gestion des conteneurs
- GitLens — suivi des modifications

3. Exercice 1 — API REST avec Flask/FastAPI

Dans cet exercice, vous allez créer une API REST complète. Une API REST (Representational State Transfer) est une architecture qui utilise les méthodes HTTP standard (GET, POST, PUT, DELETE) pour exposer des ressources.

Méthode HTTP	Action CRUD	Description
GET	Read (Lire)	Récupérer une ou plusieurs ressources
POST	Create (Créer)	Créer une nouvelle ressource
PUT	Update (Modifier)	Mettre à jour une ressource existante
DELETE	Delete (Supprimer)	Supprimer une ressource

3.A — Exemple guidé : Gestion d'une bibliothèque (Flask)

L'exemple suivant implémente une API REST complète pour gérer une collection de livres. Lisez attentivement chaque section et les commentaires explicatifs.

Fichier : `exercice1_rest/app.py`

```
#!/usr/bin/env python3
"""
API REST - Gestion de livres
Module ECHE834 - Exemple guidé (Exercice 1)
"""

from flask import Flask, jsonify, request, abort
import os
from dotenv import load_dotenv

# Chargement des variables d'environnement depuis le fichier .env
load_dotenv()

# -----
# Initialisation de l'application Flask
# -----
app = Flask(__name__)

# Token d'authentification (en production : utiliser JWT ou OAuth2)
API_TOKEN = os.getenv("API_TOKEN", "mon_token_secret_123")

# -----
# Base de données en mémoire (simulation)
# En production : utiliser une base de données réelle (PostgreSQL, MongoDB...)
# -----
livres_db = [
    {"id": 1, "titre": "Le Petit Prince", "auteur": "A. de Saint-Exupéry",
     "annee": 1943},
    {"id": 2, "titre": "1984", "auteur": "George Orwell",
     "annee": 1949},
```

```

        {"id": 3, "titre": "L'Étranger",          "auteur": "Albert Camus",
"annee": 1942},
    ]
    next_id = 4  # Compteur pour les nouveaux IDs

# -----
# MIDDLEWARE : Vérification du token d'authentification
# -----
def verifier_token():
    """Vérifie que le token Bearer est présent et valide."""
    auth_header = request.headers.get("Authorization", "")
    if not auth_header.startswith("Bearer "):
        abort(401, description="Token manquant. Format attendu: Bearer <token>")
    token = auth_header.split(" ")[1]
    if token != API_TOKEN:
        abort(403, description="Token invalide ou expiré")

# -----
# VALIDATION des données d'entrée
# -----
def valider_livre(data, required_fields=None):
    """Valide les données d'un livre. Retourne (True, None) ou (False, message)."""
    if required_fields is None:
        required_fields = ["titre", "auteur", "annee"]
    for champ in required_fields:
        if champ not in data:
            return False, f"Champ obligatoire manquant : {champ}"
    if "annee" in data:
        if not isinstance(data["annee"], int) or not (1000 <= data["annee"] <=
2100):
            return False, "L'année doit être un entier entre 1000 et 2100"
    if "titre" in data and len(data["titre"].strip()) == 0:
        return False, "Le titre ne peut pas être vide"
    return True, None

# -----
# GESTION DES ERREURS
# -----
@app.errorhandler(400)
def bad_request(e):
    return jsonify({"erreur": "Requête invalide", "detail": str(e.description)}),
400

@app.errorhandler(401)
def unauthorized(e):
    return jsonify({"erreur": "Non autorisé", "detail": str(e.description)}), 401

@app.errorhandler(403)
def forbidden(e):
    return jsonify({"erreur": "Accès refusé", "detail": str(e.description)}), 403

@app.errorhandler(404)
def not_found(e):

```



```

        return jsonify({"erreur": "Ressource introuvable", "detail":
str(e.description)}), 404

# -----
# ROUTES - Endpoints CRUD
# -----

# GET /livres - Récupérer tous les livres
@app.route("/livres", methods=["GET"])
def get_livres():
    """Retourne la liste complète des livres."""
    verifier_token()
    # Paramètre de filtre optionnel : ?auteur=Camus
    auteur_filtre = request.args.get("auteur")
    if auteur_filtre:
        resultats = [l for l in livres_db if auteur_filtre.lower() in
l["auteur"].lower()]
        return jsonify({"livres": resultats, "total": len(resultats)}), 200
    return jsonify({"livres": livres_db, "total": len(livres_db)}), 200

# GET /livres/<id> - Récupérer un livre par son ID
@app.route("/livres/<int:livre_id>", methods=["GET"])
def get_livre(livre_id):
    """Retourne un livre spécifique par son identifiant."""
    verifier_token()
    livre = next((l for l in livres_db if l["id"] == livre_id), None)
    if livre is None:
        abort(404, description=f"Aucun livre avec l'id {livre_id}")
    return jsonify(livre), 200

# POST /livres - Créer un nouveau livre
@app.route("/livres", methods=["POST"])
def create_livre():
    """Crée un nouveau livre dans la base de données."""
    global next_id
    verifier_token()
    # Vérification du Content-Type
    if not request.is_json:
        abort(400, description="Le corps de la requête doit être en JSON")
    data = request.get_json()
    valide, message = valider_livre(data)
    if not valide:
        abort(400, description=message)
    nouveau_livre = {
        "id": next_id,
        "titre": data["titre"].strip(),
        "auteur": data["auteur"].strip(),
        "annee": data["annee"],
    }
    livres_db.append(nouveau_livre)
    next_id += 1
    # HTTP 201 Created - Bonne pratique REST
    return jsonify(nouveau_livre), 201

```

```

# PUT /livres/<id> - Mettre à jour un livre
@app.route("/livres/<int:livre_id>", methods=["PUT"])
def update_livre(livre_id):
    """Met à jour un livre existant (remplacement complet)."""
    verifier_token()
    livre = next((l for l in livres_db if l["id"] == livre_id), None)
    if livre is None:
        abort(404, description=f"Aucun livre avec l'id {livre_id}")
    if not request.is_json:
        abort(400, description="Le corps de la requête doit être en JSON")
    data = request.get_json()
    valide, message = valider_livre(data)
    if not valide:
        abort(400, description=message)
    livre["titre"] = data["titre"].strip()
    livre["auteur"] = data["auteur"].strip()
    livre["annee"] = data["annee"]
    return jsonify(livre), 200

# DELETE /livres/<id> - Supprimer un livre
@app.route("/livres/<int:livre_id>", methods=["DELETE"])
def delete_livre(livre_id):
    """Supprime un livre de la base de données."""
    global livres_db
    verifier_token()
    livre = next((l for l in livres_db if l["id"] == livre_id), None)
    if livre is None:
        abort(404, description=f"Aucun livre avec l'id {livre_id}")
    livres_db = [l for l in livres_db if l["id"] != livre_id]
    # HTTP 204 No Content - Bonne pratique REST pour DELETE
    return "", 204

# -----
# Point d'entrée de l'application
# -----
if __name__ == "__main__":
    print("=== API Livres démarrée sur http://localhost:5000 ===")
    app.run(debug=True, host="0.0.0.0", port=5000)

```

Démarrage et test rapide

Pour lancer l'API et la tester :

```

# Lancer l'API
python app.py

# Tester avec curl (dans un autre terminal)
# 1. GET tous les livres
curl -H "Authorization: Bearer mon_token_secret_123" http://localhost:5000/livres

# 2. POST - Créer un livre
curl -X POST http://localhost:5000/livres \
  -H "Authorization: Bearer mon_token_secret_123" \
  -H "Content-Type: application/json" \

```

```

    -d '{"titre": "Dune", "auteur": "Frank Herbert", "annee": 1965}'

# 3. PUT – Modifier un livre (id=1)
curl -X PUT http://localhost:5000/livres/1 \
  -H "Authorization: Bearer mon_token_secret_123" \
  -H "Content-Type: application/json" \
  -d '{"titre": "Le Petit Prince", "auteur": "Saint-Exupéry", "annee": 1943}'

# 4. DELETE – Supprimer un livre
curl -X DELETE http://localhost:5000/livres/2 \
  -H "Authorization: Bearer mon_token_secret_123"

```

🔑 **Codes de statut HTTP à retenir** : 200 OK (lecture/modification réussie), 201 Created (création réussie), 204 No Content (suppression réussie), 400 Bad Request (données invalides), 401 Unauthorized (token manquant), 403 Forbidden (token invalide), 404 Not Found (ressource inexistante).

3.B — Exercice pratique : Créez votre propre API REST

📋 **Consigne** : En vous basant sur l'exemple ci-dessus, créez une API REST complète pour gérer l'un des domaines suivants (au choix) :

Gestion d'événements (nom, lieu, date, capacité_max, organisateur)

Votre API doit implémenter les éléments suivants :

Critère	Description
Routes CRUD	GET (liste + détail), POST, PUT, DELETE
Authentification	Vérification d'un token Bearer sur chaque route
Validation	Valider les champs obligatoires et leurs types
Gestion des erreurs	Retourner les codes HTTP appropriés (400, 401, 404...)
Filtre optionnel	Au moins un paramètre de requête (query param) pour filtrer
Code commenté	Chaque section doit être commentée en français

Structure de fichier attendue :

```

exercicel_rest/
├─ app.py           # Application principale
├─ .env             # Variables d'environnement (API_TOKEN=...)
├─ tests/
│   └─ test_api.py # Tests pytest (cf. section 3.C)

```

3.C — Tests avec pytest

Les tests permettent de s'assurer que votre API fonctionne correctement et qu'elle continue de fonctionner après des modifications. Voici un exemple de tests pour l'API livres :

Fichier : exercice1_rest/tests/test_api.py

```
"""
Tests unitaires et d'intégration pour l'API REST Livres
Lancement : pytest tests/ -v
"""

import pytest
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from app import app, livres_db, next_id

TOKEN = "mon_token_secret_123"
HEADERS_OK = {"Authorization": f"Bearer {TOKEN}"}
HEADERS_BAD = {"Authorization": "Bearer mauvais_token"}

@pytest.fixture
def client():
    """Fixture : crée un client de test Flask."""
    app.config["TESTING"] = True
    with app.test_client() as client:
        yield client

# — Tests GET —————
class TestGetLivres:
    def test_get_tous_les_livres(self, client):
        """GET /livres → 200 + liste non vide."""
        r = client.get("/livres", headers=HEADERS_OK)
        assert r.status_code == 200
        data = r.get_json()
        assert "livres" in data
        assert data["total"] >= 3

    def test_get_livre_par_id(self, client):
        """GET /livres/1 → 200 + titre correct."""
        r = client.get("/livres/1", headers=HEADERS_OK)
        assert r.status_code == 200
        assert r.get_json()["titre"] == "Le Petit Prince"

    def test_get_livre_inexistant(self, client):
        """GET /livres/999 → 404."""
        r = client.get("/livres/999", headers=HEADERS_OK)
        assert r.status_code == 404

    def test_get_sans_token(self, client):
        """GET /livres sans token → 401."""
        r = client.get("/livres")
        assert r.status_code == 401

    def test_get_mauvais_token(self, client):
        """GET /livres avec mauvais token → 403."""
        r = client.get("/livres", headers=HEADERS_BAD)
```

```

        assert r.status_code == 403

# — Tests POST —————
class TestCreateLivre:
    def test_creer_livre_valide(self, client):
        """POST /livres avec données valides → 201."""
        payload = {"titre": "Dune", "auteur": "Frank Herbert", "annee": 1965}
        r = client.post("/livres", json=payload, headers=HEADERS_OK)
        assert r.status_code == 201
        livre = r.get_json()
        assert livre["titre"] == "Dune"
        assert "id" in livre

    def test_creer_livre_champ_manquant(self, client):
        """POST /livres sans auteur → 400."""
        payload = {"titre": "Livre sans auteur", "annee": 2020}
        r = client.post("/livres", json=payload, headers=HEADERS_OK)
        assert r.status_code == 400

    def test_creer_livre_annee_invalide(self, client):
        """POST /livres avec année invalide → 400."""
        payload = {"titre": "Test", "auteur": "Test", "annee": 99}
        r = client.post("/livres", json=payload, headers=HEADERS_OK)
        assert r.status_code == 400

# — Tests DELETE —————
class TestDeleteLivre:
    def test_supprimer_livre(self, client):
        """DELETE /livres/1 → 204."""
        r = client.delete("/livres/1", headers=HEADERS_OK)
        assert r.status_code == 204

    def test_supprimer_livre_inexistant(self, client):
        """DELETE /livres/999 → 404."""
        r = client.delete("/livres/999", headers=HEADERS_OK)
        assert r.status_code == 404

```

Lancer les tests

```

# Lancer tous les tests avec sortie détaillée
pytest tests/ -v

# Lancer avec rapport de couverture
pip install pytest-cov
pytest tests/ -v --cov=. --cov-report=term-missing

```

3.D — Critères d'évaluation (Exercice 1)

Critère	Points	Description
Routes CRUD complètes	/5	GET (liste+détail), POST, PUT, DELETE fonctionnels
Authentification	/3	Token vérifié sur chaque route,

		erreurs 401/403
Validation & erreurs	/4	Données validées, codes HTTP corrects
Qualité du code	/4	Code commenté, structure claire, bonnes pratiques
Tests pytest	/4	Tests couvrant les cas nominaux et les cas d'erreur

4. Exercice 2 — API GraphQL avec Strawberry

GraphQL est un langage de requête pour les APIs développé par Facebook. Contrairement à REST où chaque endpoint retourne une structure fixe, GraphQL permet au client de demander exactement les champs dont il a besoin. Nous utilisons ici la bibliothèque Strawberry, qui exploite les type hints Python.

Aspect	REST	GraphQL
Endpoint	Un endpoint par ressource (/livres, /livres/1)	Un seul endpoint /graphql
Données	Structure de réponse fixée par le serveur	Le client choisit les champs
Versioning	Versions d'API (/v1/, /v2/)	Évolution du schéma sans version
Apprentissage	Facile, intuitif	Courbe d'apprentissage plus élevée

4.A — Exemple guidé : Gestion de livres (Strawberry + FastAPI)

Le même domaine (livres) est utilisé pour faciliter la comparaison avec l'exercice REST. L'API GraphQL expose les mêmes fonctionnalités via queries et mutations.

Fichier : `exercice2_graphql/schema.py`

```
"""
Schéma GraphQL - Gestion de livres
Module ECHE834 - Exemple guidé (Exercice 2)
"""

import strawberry
from typing import Optional, List

# -----
# TYPES GraphQL
# Les types définissent la structure des données exposées par l'API
# -----

@strawberry.type
class Livre:
    """Représente un livre dans notre système."""
    id: int
    titre: str
    auteur: str
    annee: int

@strawberry.input
class LivreInput:
    """Données d'entrée pour créer ou modifier un livre."""
    titre: str
    auteur: str
    annee: int

@strawberry.type
```

```

class DeleteResult:
    """Résultat d'une suppression."""
    success: bool
    message: str

# -----
# BASE DE DONNÉES EN MÉMOIRE
# -----
livres_db: List[dict] = [
    {"id": 1, "titre": "Le Petit Prince",      "auteur": "A. de Saint-Exupéry",
"annee": 1943},
    {"id": 2, "titre": "1984",                  "auteur": "George Orwell",
"annee": 1949},
    {"id": 3, "titre": "L'Étranger",            "auteur": "Albert Camus",
"annee": 1942},
]
_next_id = 4

def dict_to_livre(d: dict) -> Livre:
    """Convertit un dictionnaire en objet Livre."""
    return Livre(id=d["id"], titre=d["titre"], auteur=d["auteur"], annee=d["annee"])

# -----
# QUERIES – Lecture de données (équivalent GET en REST)
# -----

@strawberry.type
class Query:

    @strawberry.field(description="Retourne tous les livres (filtre optionnel par
auteur)")
    def livres(self, auteur: Optional[str] = None) -> List[Livre]:
        if auteur:
            return [dict_to_livre(l) for l in livres_db
                    if auteur.lower() in l["auteur"].lower()]
        return [dict_to_livre(l) for l in livres_db]

    @strawberry.field(description="Retourne un livre par son identifiant")
    def livre(self, id: int) -> Optional[Livre]:
        livre = next((l for l in livres_db if l["id"] == id), None)
        return dict_to_livre(livre) if livre else None

# -----
# MUTATIONS – Modification de données (équivalent POST/PUT/DELETE en REST)
# -----

@strawberry.type
class Mutation:

    @strawberry.mutation(description="Crée un nouveau livre")
    def creer_livre(self, livre_input: LivreInput) -> Livre:
        global _next_id
        # Validation

```



```

        if not (1000 <= livre_input.annee <= 2100):
            raise ValueError("L'année doit être comprise entre 1000 et 2100")
        if not livre_input.titre.strip():
            raise ValueError("Le titre ne peut pas être vide")
        nouveau = {
            "id": _next_id,
            "titre": livre_input.titre.strip(),
            "auteur": livre_input.auteur.strip(),
            "annee": livre_input.annee,
        }
        livres_db.append(nouveau)
        _next_id += 1
        return dict_to_livre(nouveau)

@strawberry.mutation(description="Modifie un livre existant")
def modifier_livre(self, id: int, livre_input: LivreInput) -> Optional[Livre]:
    livre = next((l for l in livres_db if l["id"] == id), None)
    if livre is None:
        return None
    livre["titre"] = livre_input.titre.strip()
    livre["auteur"] = livre_input.auteur.strip()
    livre["annee"] = livre_input.annee
    return dict_to_livre(livre)

@strawberry.mutation(description="Supprime un livre")
def supprimer_livre(self, id: int) -> DeleteResult:
    global livres_db
    avant = len(livres_db)
    livres_db = [l for l in livres_db if l["id"] != id]
    if len(livres_db) < avant:
        return DeleteResult(success=True, message=f"Livre {id} supprimé")
    return DeleteResult(success=False, message=f"Livre {id} introuvable")

# -----
# SCHÉMA final - combine Query et Mutation
# -----
schema = strawberry.Schema(query=Query, mutation=Mutation)

```

Fichier : exercice2_graphql/app.py

```

"""
Application FastAPI + GraphQL (Strawberry)
Module ECHE834 - Exemple guidé (Exercice 2)
"""

from fastapi import FastAPI, Depends, HTTPException, Header
from strawberry.fastapi import GraphQLRouter
from schema import schema
import os
from dotenv import load_dotenv

load_dotenv()
API_TOKEN = os.getenv("API_TOKEN", "mon_token_secret_123")

```

```

app = FastAPI(title="API GraphQL - Livres", version="1.0.0")

async def verifier_token(authorization: str = Header(...)):
    """Dépendance FastAPI pour la vérification du token."""
    if not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Format: Bearer <token>")
    token = authorization.split(" ")[1]
    if token != API_TOKEN:
        raise HTTPException(status_code=403, detail="Token invalide")

# Routeur GraphQL - accessible via /graphql
graphql_router = GraphQLRouter(
    schema,
    dependencies=[Depends(verifier_token)]
)
app.include_router(graphql_router, prefix="/graphql")

@app.get("/")
async def root():
    return {"message": "API GraphQL opérationnelle", "docs": "/graphql"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app:app", host="0.0.0.0", port=8000, reload=True)

```

Démarrage et exemples de requêtes GraphQL

```

# Lancer l'API GraphQL
uvicorn app:app --reload --port 8000

# L'interface GraphiQL est disponible dans le navigateur :
# http://localhost:8000/graphql

# — Exemples de requêtes GraphQL —————

# QUERY - Récupérer tous les livres (seulement titre et auteur)
query {
  livres {
    titre
    auteur
  }
}

# QUERY - Récupérer un livre par ID
query {
  livre(id: 1) {
    id
    titre
    auteur
    annee
  }
}

```

```

}

# MUTATION - Créer un nouveau livre
mutation {
  creerLivre(livreInput: {
    titre: "Fondation"
    auteur: "Isaac Asimov"
    annee: 1951
  }) {
    id
    titre
  }
}

# MUTATION - Supprimer un livre
mutation {
  supprimerLivre(id: 2) {
    success
    message
  }
}

# Via curl :
curl -X POST http://localhost:8000/graphql \
  -H "Authorization: Bearer mon_token_secret_123" \
  -H "Content-Type: application/json" \
  -d '{"query": "{ livres { titre auteur } }"}'
```

4.B — Exercice pratique : Créez votre API GraphQL

 **Consigne :** Reprenez le même domaine que votre exercice REST (exercice 1.B) et réimplémentez-le en GraphQL avec Strawberry. Cela vous permettra de comparer les deux approches sur un exemple que vous connaissez déjà.

Votre API GraphQL doit implémenter :

- Un type GraphQL correspondant à votre ressource (avec au moins 4 champs)
- Un type Input pour les créations/modifications
- Au moins 2 Queries : liste et récupération par ID
- Au moins 3 Mutations : créer, modifier, supprimer
- Validation des données dans les mutations
- Authentification par token (Bearer)
- Un fichier app.py qui expose l'API via FastAPI sur /graphql

4.C — Tests d'une API GraphQL

```

"""
Tests pour l'API GraphQL avec pytest + httpx (client async)
Lancement : pytest tests/ -v
"""
```

```

import pytest
from fastapi.testclient import TestClient
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from app import app

TOKEN = "mon_token_secret_123"
HEADERS = {"Authorization": f"Bearer {TOKEN}"}
URL = "/graphql"

@pytest.fixture
def client():
    return TestClient(app)

def gql(client, query, variables=None, headers=None):
    """Helper : exécute une requête GraphQL."""
    payload = {"query": query}
    if variables:
        payload["variables"] = variables
    return client.post(URL, json=payload, headers=headers or HEADERS)

class TestQueries:
    def test_lister_livres(self, client):
        q = "{ livres { id titre auteur } }"
        r = gql(client, q)
        assert r.status_code == 200
        data = r.json()["data"]
        assert len(data["livres"]) >= 3

    def test_livre_par_id(self, client):
        q = "{ livre(id: 1) { titre } }"
        r = gql(client, q)
        assert r.status_code == 200
        assert r.json()["data"]["livre"]["titre"] == "Le Petit Prince"

    def test_livre_inexistant(self, client):
        q = "{ livre(id: 999) { titre } }"
        r = gql(client, q)
        # GraphQL retourne 200 avec data.livre = null
        assert r.json()["data"]["livre"] is None

class TestMutations:
    def test_creer_livre(self, client):
        mutation = """
        mutation {
            creerLivre(livreInput: {titre: "Test", auteur: "Auteur", annee: 2024}) {
                id titre
            }
        }
        """
        r = gql(client, mutation)

```

```

assert r.status_code == 200
livre = r.json()["data"]["creerLivre"]
assert livre["titre"] == "Test"
assert "id" in livre

def test_supprimer_livre(self, client):
    mutation = "mutation { supprimerLivre(id: 1) { success message } }"
    r = gql(client, mutation)
    assert r.json()["data"]["supprimerLivre"]["success"] is True

def test_sans_token(self, client):
    r = client.post(URL, json={"query": "{ livres { titre } }"})
    # FastAPI retourne 422 si le header Authorization est absent
    assert r.status_code in [401, 403, 422]

```

4.D — Critères d'évaluation (Exercice 2)

Critère	Points	Description
Schéma GraphQL	/4	Types, Input, DeleteResult correctement définis
Queries	/3	Liste + détail fonctionnels
Mutations	/4	Créer, modifier, supprimer fonctionnels
Authentification	/3	Token Bearer vérifié
Tests pytest	/6	Couverture queries, mutations, cas d'erreur

5. Guide de Dockerisation

Docker permet de packager une application et toutes ses dépendances dans un conteneur portable. Cela garantit que l'API fonctionnera de la même façon en développement, en test et en production, quel que soit le système d'exploitation.

5.1 Concepts clés Docker

Concept	Définition
Image	Template immuable contenant l'OS, le runtime Python et le code
Conteneur	Instance en cours d'exécution d'une image
Dockerfile	Fichier de recette pour construire une image
docker-compose	Outil pour orchestrer plusieurs conteneurs
Volume	Stockage persistant partagé entre l'hôte et le conteneur
Port mapping	Lien entre un port du conteneur et un port de la machine hôte

5.2 Dockerfile — API REST (Flask)

```
# — Dockerfile (API REST Flask) —

# Image de base : Python 3.11 slim (version légère)
FROM python:3.11-slim

# Définir le répertoire de travail dans le conteneur
WORKDIR /app

# Copier uniquement requirements.txt en premier (optimisation du cache)
# Si requirements.txt ne change pas, Docker réutilise la couche du cache
COPY requirements.txt .

# Installer les dépendances (sans cache pip pour réduire la taille)
RUN pip install --no-cache-dir -r requirements.txt

# Copier le reste du code source
COPY . .

# Exposer le port sur lequel Flask écoute
EXPOSE 5000

# Variables d'environnement (peuvent être surchargées au lancement)
ENV FLASK_ENV=production
ENV API_TOKEN=mon_token_secret_123

# Commande de démarrage (utiliser gunicorn en production)
CMD ["python", "app.py"]
# En production : CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:app"]
```

5.3 Dockerfile — API GraphQL (FastAPI)

```
# — Dockerfile (API GraphQL FastAPI) —  
  
FROM python:3.11-slim  
  
WORKDIR /app  
  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY . .  
  
EXPOSE 8000  
  
ENV API_TOKEN=mon_token_secret_123  
  
# Uvicorn : serveur ASGI pour FastAPI  
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

5.4 docker-compose.yml — Orchestration des deux APIs

```
# — docker-compose.yml —  
# Lance les deux APIs en même temps avec une commande unique  
  
version: "3.9"  
  
services:  
  
  # API REST Flask  
  api-rest:  
    build:  
      context: ./exercice1_rest  
      dockerfile: Dockerfile  
    container_name: eche834-api-rest  
    ports:  
      - "5000:5000"          # hôte:conteneur  
    environment:  
      - API_TOKEN=${API_TOKEN:-mon_token_secret}  
      - FLASK_ENV=development  
    volumes:  
      - ./exercice1_rest:/app # Hot reload en développement  
    restart: unless-stopped  
  
  # API GraphQL FastAPI  
  api-graphql:  
    build:  
      context: ./exercice2_graphql  
      dockerfile: Dockerfile  
    container_name: eche834-api-graphql  
    ports:  
      - "8000:8000"
```

```

environment:
  - API_TOKEN=${API_TOKEN:-mon_token_secret}
volumes:
  - ./exercice2_graphql:/app
restart: unless-stopped
# Démarre uniquement après l'API REST
depends_on:
  - api-rest

# Les deux services partagent le même réseau (par défaut)
# Ils peuvent se contacter via leurs noms de service :
# http://api-rest:5000 et http://api-graphql:8000

```

5.5 Commandes Docker essentielles

```

# ——— Commandes de base ———

# Construire une image
docker build -t eche834-api-rest ./exercicel_rest

# Lancer un conteneur
docker run -p 5000:5000 -e API_TOKEN=secret eche834-api-rest

# ——— Avec docker-compose ———

# Lancer tous les services (en arrière-plan)
docker-compose up -d

# Voir les logs en temps réel
docker-compose logs -f

# Voir les logs d'un service spécifique
docker-compose logs -f api-rest

# Arrêter tous les services
docker-compose down

# Reconstruire les images après modification du code
docker-compose up -d --build

# Exécuter une commande dans un conteneur en cours d'exécution
docker exec -it eche834-api-rest bash

# Voir l'état des conteneurs
docker-compose ps

# Lancer les tests à l'intérieur du conteneur
docker exec eche834-api-rest pytest tests/ -v

```

 **Bonne pratique :** Ne jamais mettre de secrets (tokens, mots de passe) directement dans le Dockerfile

ou dans docker-compose.yml. Utilisez un fichier .env (non commité dans git) et la syntaxe `${MA_VARIABLE:-valeur_par_defaut}`.

6. Bonnes pratiques générales

6.1 Architecture du projet

Un projet API bien structuré respecte le principe de séparation des responsabilités :

```
mon_api/
├── app.py           # Point d'entrée, configuration
├── routes.py        # Définition des endpoints (Flask)
├── models.py        # Modèles de données / schémas Pydantic
├── services.py      # Logique métier (traitement, calculs)
├── repositories.py  # Accès aux données (DB, fichiers)
├── config.py        # Configuration (lecture des variables d'env)
├── .env             # Variables d'environnement (NE PAS commiter)
├── .gitignore       # Exclure venv/, .env, __pycache__/
└── tests/
    ├── test_routes.py
    └── test_services.py
```

6.2 Gestion des configurations

Utilisez toujours des variables d'environnement pour les valeurs sensibles ou qui changent selon l'environnement :

```
# Fichier .env (NE PAS commiter dans git !)
API_TOKEN=mon_token_tres_secret
DATABASE_URL=postgresql://user:password@localhost/mydb
DEBUG=False
PORT=5000

# Fichier config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    API_TOKEN = os.getenv("API_TOKEN", "changez_moi_en_prod")
    DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///dev.db")
    DEBUG = os.getenv("DEBUG", "False").lower() == "true"
    PORT = int(os.getenv("PORT", "5000"))

# Fichier .gitignore
venv/
.env
__pycache__/
*.pyc
.pytest_cache/
```

6.3 Sécurité

Aspect	Recommandation
Authentification	Utiliser JWT (JSON Web Tokens) en production plutôt que des tokens statiques. Bibliothèque : <code>python-jose</code> ou <code>PyJWT</code> .
HTTPS	Ne jamais déployer une API en HTTP. Utiliser un reverse proxy (Nginx) avec certificat SSL.
Validation	Valider toutes les données entrantes (types, longueurs, formats). Avec FastAPI, Pydantic le fait automatiquement.
Rate limiting	Limiter le nombre de requêtes par IP/token pour éviter les abus (<code>flask-limiter</code> , <code>slowapi</code> pour FastAPI).
CORS	Configurer strictement les origines autorisées (<code>flask-cors</code> , <code>FastAPI CORSMiddleware</code>).

6.4 Documentation automatique (Swagger/OpenAPI)

FastAPI génère automatiquement une documentation interactive :

```
# FastAPI génère automatiquement deux interfaces de documentation :
# • Swagger UI : http://localhost:8000/docs
# • ReDoc      : http://localhost:8000/redoc

# Enrichir la documentation avec des descriptions :
from fastapi import FastAPI

app = FastAPI(
    title="API Livres ECHE834",
    description="API REST de gestion de livres - TD ECHE834",
    version="1.0.0",
    contact={"name": "ECHE834", "email": "contact@exemple.fr"},
)

# Documenter un endpoint :
@app.get("/livres", summary="Lister les livres", tags=["Livres"])
async def get_livres():
    """
    Retourne la liste complète des livres.

    - **auteur** : filtre optionnel par nom d'auteur
    """
    ...
```

6.5 Logging et monitoring

```
import logging
import time
from flask import Flask, request, g

# Configuration du logging
logging.basicConfig(
    level=logging.INFO,
```

```

    format="%(asctime)s [%(levelname)s] %(name)s - %(message)s",
    handlers=[
        logging.StreamHandler(),          # Console
        logging.FileHandler("api.log"),   # Fichier
    ]
)
logger = logging.getLogger(__name__)

app = Flask(__name__)

# Middleware : logger chaque requête entrante
@app.before_request
def before_request():
    g.start_time = time.time()
    logger.info(f"→ {request.method} {request.path}")

@app.after_request
def after_request(response):
    duration = round((time.time() - g.start_time) * 1000, 2)
    logger.info(f"← {response.status_code} [{duration}ms]")
    return response

```

7. Annexes

7.1 Ressources complémentaires

Ressource	Lien / Accès
Documentation Flask	https://flask.palletsprojects.com/
Documentation FastAPI	https://fastapi.tiangolo.com/
Documentation Strawberry	https://strawberry.rocks/
Documentation pytest	https://docs.pytest.org/
Documentation Docker	https://docs.docker.com/
REST API Best Practices	https://aws.amazon.com/fr/startups/start-building/how-to-build-a-public-facing-API/
Introduction GraphQL	https://graphql.org/learn/
Pydantic (validation)	https://docs.pydantic.dev/
python-dotenv	https://pypi.org/project/python-dotenv/
360Learning / ENI	Bibliothèques numériques de l'école

7.2 Commandes de référence rapide

```
# — Python / pip —
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
pip freeze > requirements.txt

# — Flask —
FLASK_APP=app.py FLASK_DEBUG=1 flask run --port 5000

# — FastAPI / Uvicorn —
uvicorn app:app --reload --port 8000

# — Tests —
pytest tests/ -v # tests verbeux
pytest tests/ -v --cov=. --cov-report=term-missing # avec couverture
pytest tests/test_api.py::TestGetLivres # classe spécifique

# — Docker —
docker build -t mon-api . # construire l'image
docker run -p 5000:5000 mon-api # lancer le conteneur
docker-compose up -d # lancer tous les services
docker-compose down # arrêter tous les services
docker-compose logs -f # voir les logs
docker ps # conteneurs en cours
docker images # images disponibles

# — curl — test rapide d'une API —
curl -s http://localhost:5000/livres \
  -H "Authorization: Bearer mon_token" | python3 -m json.tool
```

7.3 Troubleshooting — Problèmes courants

Problème	Cause probable	Solution
----------	----------------	----------

ModuleNotFoundError	Paquet non installé ou venv non activé	Vérifier que (venv) est actif. Relancer pip install -r requirements.txt
Port déjà utilisé (EADDRINUSE)	Un autre processus écoute sur le même port	Tuer le processus : kill \$(lsof -ti:5000) ou changer le port
401 Unauthorized	Token absent de la requête	Ajouter le header : Authorization: Bearer <votre_token>
403 Forbidden	Token incorrect	Vérifier la valeur du token dans .env et dans la requête
400 Bad Request	Données manquantes ou invalides	Vérifier le Content-Type (application/json) et les champs envoyés
Docker : image not found	Image non construite	Lancer docker build -t nom-image . avant docker run

7.4 Exemple de fichier .env

```
# Fichier .env - Variables d'environnement
# ⚠ NE PAS commiter ce fichier dans git (ajouter .env dans .gitignore)

# Token d'authentification API
API_TOKEN=mon_token_tres_secret_a_changer_en_prod

# Configuration Flask
FLASK_ENV=development
FLASK_DEBUG=1

# Port de l'application
PORT=5000

# URL de base de données (si applicable)
# DATABASE_URL=postgresql://user:password@localhost:5432/mydb
```